

Algorithms Problem Solving

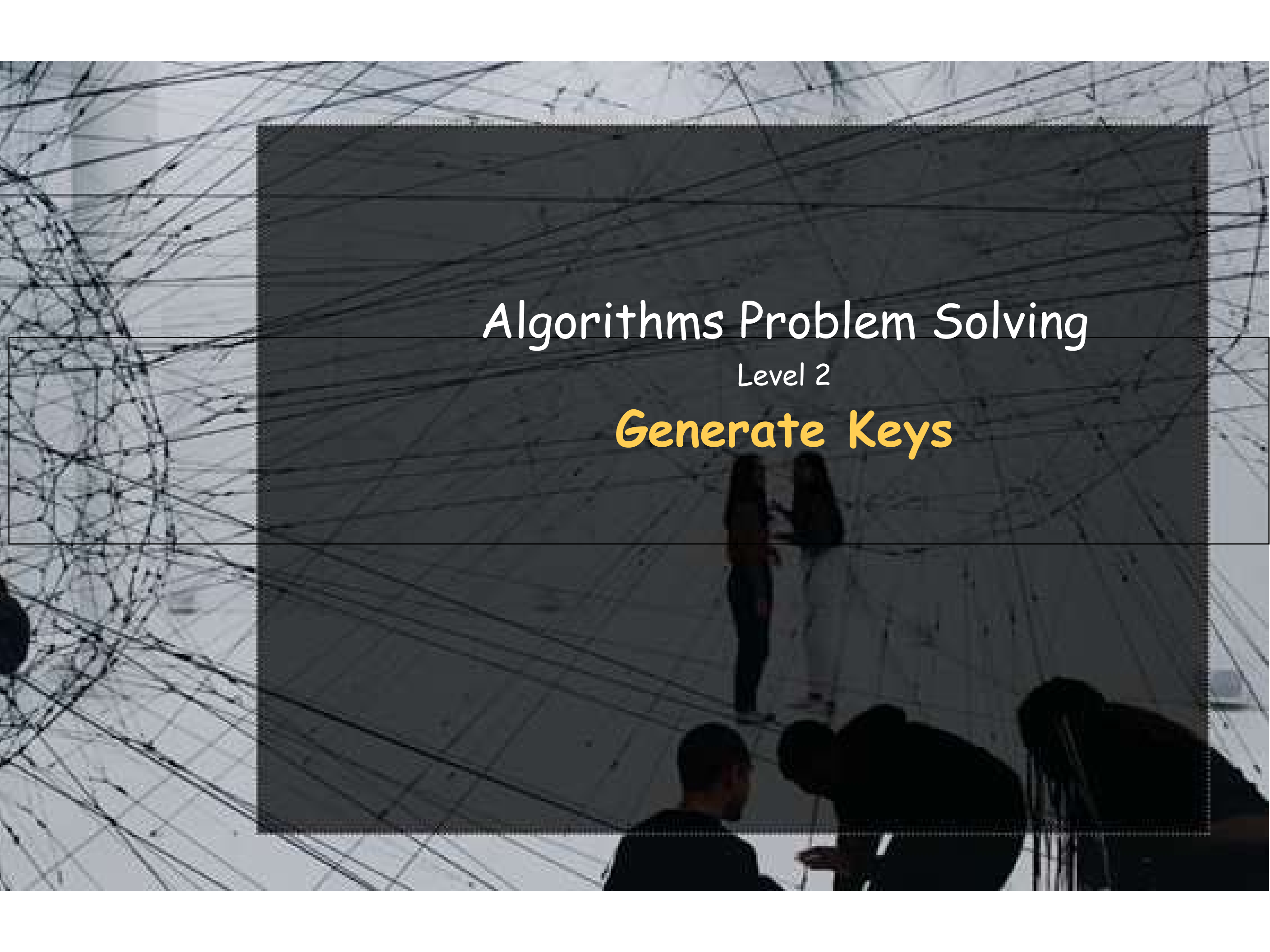
Level 2

Generate Keys

Algorithms Problem Solving

Level 2

Generate Keys



Definition

- ◆ A program that asks the user how many keys to generate and then prints that number of random keys.
- ◆ Each key contains 4 groups of 4 random capital letters separated by `-`.
- ◆ The format of each key is:
`XXXX-XXXX-XXXX-XXXX`

Explanation

- ◆ ``ReadNumber()`` asks the user how many keys should be generated and ensures the number is greater than ``0``.
- ◆ ``GenerateWord()`` creates a string containing a specified number of random characters of a selected type.
- ◆ ``GenerateKey()`` creates one complete key by generating four groups of four capital letters.
- ◆ Each group is generated using ``GenerateWord(enCharType::CapitalLetter, 4)``.
- ◆ The groups are connected using ``-``.
- ◆ ``GenerateKeys()`` repeats the key-generation process according to the number entered by the user.
- ◆ ``RandomNumber()`` generates a random value within the required character range.
- ◆ ``GetRandomCharacter()`` uses the ``enum`` and ``switch`` to generate the requested character type.

Steps in main()

1. Initialize the random number generator using `srand()`.
2. Ask the user how many keys to generate.
3. Validate that the number is greater than `0`.
4. Generate the first group of 4 capital letters.
5. Add `-`.
6. Generate the second group of 4 capital letters.
7. Add `-`.
8. Generate the third group of 4 capital letters.
9. Add `-`.
10. Generate the fourth group of 4 capital letters.
11. Print the complete key.
12. Repeat until the requested number of keys has been generated.

💡 Example

◆ Input:
5

◆ One possible output is:

Key [1] : GHYF-YGTL-CZZH-WAIC

Key [2] : AUYH-OZYZ-ONFH-CSFX

Key [3] : GQUH-SOXW-SUAI-DPLX

Key [4] : LLWJ-KVNF-KHGU-ABQC

Key [5] : GRMK-UBZK-DEGV-BDZV

◆ Every execution can produce different keys because the characters are randomly generated.

➡ Result

◆ If the user enters `5`, the program generates 5 random keys.

◆ Each key contains:

 `4 + 4 + 4 + 4 = 16` capital letters

◆ There are also 3 separators `-`.

◆ Therefore, each key has 19 characters in total.

✚ Note

- ◆ ``GenerateWord()`` is a reusable function that can generate a string of any requested length.
- ◆ For example:
 - ``GenerateWord(CapitalLetter, 4)`` → ``GHYF``
- ◆ ``GenerateKey()`` uses this function four times to build the complete key.
- ◆ The expression:
 - ``key = GenerateWord(...) + "-";``
starts the key with the first group and a separator.
- ◆ Then:
 - ``key = key + GenerateWord(...) + "-";``
adds another group and separator.
- ◆ This is an example of breaking a larger task into smaller reusable functions.

⊘ Warning

- ◆ The generated keys are random-looking but should not be considered secure keys for real authentication or encryption systems because `rand()` is not designed for cryptographic security.
- ◆ The program only generates capital letters even though `GetRandomCharacter()` supports other character types.
- ◆ The `break` statements after `return` in `GetRandomCharacter()` are unreachable and can be removed.
- ◆ The code uses `rand()`, `srand()`, and `time()` but does not explicitly include `<cstdlib>` and `<ctime>`.

Comparison

- `GenerateWord()` → generates a group of random characters.
- `GenerateKey()` → combines four groups into one complete key.
- `GenerateKeys()` → generates multiple complete keys.
- `RandomNumber()` → generates a random numeric value within a range.
- `GetRandomCharacter()` → converts the random value into the requested character type.

✦ Summary

- ✓ The program generates a user-specified number of random keys.
- ✓ Each key has the format `XXXX-XXXX-XXXX-XXXX`.
- ✓ Each group contains 4 random capital letters.
- ✓ `GenerateWord()` creates each group.
- ✓ `GenerateKey()` creates one complete key.
- ✓ `GenerateKeys()` creates multiple keys.
- ✓ Functions are combined together to solve a larger problem.
- ✓ `rand()` and `srand()` are used to generate pseudo-random characters.

Thank you 😊